

Informe Trabajo Práctico N°1

W-UNGS-dle

Lizarraga Jorge

Lucero Juan

Dino Martin

Mancilla Diego

Introducción

El trabajo práctico consiste en una implementación del juego Wordle, en el cual el usuario debe adivinar una palabra oculta en una cantidad limitada de intentos. El objetivo principal del programa es simular la lógica del juego original, permitiendo la interacción del usuario mediante una interfaz y aplicando reglas de validación sobre cada intento.

Estructura del proyecto y diseño

El proyecto está organizado en distintos paquetes, separando la lógica del juego de la interfaz gráfica y los recursos utilizados.

Organización general del proyecto

- **entidades:** contiene la lógica del juego y los datos a manejar
- **gui:** contiene todas las interfaces gráficas
- **principal:** donde se ejecuta el programa
- **recursos:** archivos .txt con palabras y imágenes

Clases principales del proyecto

Clase **ConfiguracionInicial**

Inicializa el juego

```
public void crearConfiguracionInicial(Wungsdle wordle) {  
  
    wungsdle = wordle;  
    wungsdle.crearPalabra("Español - ES", "Facil - Easy");  
    System.out.println(wungsdle.getPalabraSecreta());  
    InterfazInicio window = new InterfazInicio(wungsdle);  
    window.setVisible(true);  
    window.setLocationRelativeTo(null);  
}
```

Encargada de crear la instancia principal del juego, generar palabra inicial y lanzar la interfaz principal

Clase **Palabra**

Encapsula la palabra secreta del juego

```
private String palabra;  
  
public Palabra()  
{  
    this.palabra = "";  
}  
public Palabra(String palabra)  
{  
    this.palabra = palabra;  
}  
public boolean compararPalabra(String palabra)  
{  
    return palabra.equals(palabra);  
}
```

Encargada de almacenar la palabra actual para permitir comparaciones con el usuario

Clase **Ranking**

Gestiona el almacenamiento de puntajes en un archivo.txt

```
public static void guardarPuntaje(String nombre, int puntos, int tiempo) {  
    File archivo = obtenerArchivo();  
  
    try (FileWriter fw = new FileWriter(archivo, true)) {  
        fw.write(nombre + "," + puntos + "," + tiempo + "\n");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

Encargada también de leer el puntaje, ordenar los puntos, el tiempo y reiniciar el Ranking

Clase **Usuario**

Encapsula la palabra secreta del juego Representa al jugador y almacena su información

```

public Usuario()
{
    this.nombre = "";
    this.tiempoRespuesta = 0L;
    this.intento = 6;
    this.puntos = 0;
}

public Usuario(String nombre)
{
    this.nombre = nombre;
    this.tiempoRespuesta = 0L;
    this.intento = 6;
    this.puntos = 0;
}

```

Encargada también de guardar el nombre del jugador, controlar la cantidad de intentos, acumular puntos y registrar el tiempo de juego

Clase **Wungsdle**

La clase más importante del proyecto

```

public void crearPalabra(String idiomaElegido, String dificultadElegida) {
    String rutaRecurso = rutaTxtSegunSeleccion(idiomaElegido, dificultadElegida);
    palabra.setPalabra(palabraAleatoriaDesdeRecurso(rutaRecurso));
    //Acá sigilosamente me robo la dificultad
    dificultadActual=dificultadElegida;
}

```

```

public String[] evaluarColorLetra(String intento)
{
    palabraUsuario = intento;
    String palabraSecreta = palabra.getPalabra();
    String[] resultado = new String[5];

    for (int i = 0; i < 5; i++)
    {
        char letraIntento = intento.charAt(i);
        char letraSecreta = palabraSecreta.charAt(i);

        if (letraIntento == letraSecreta)
        {
            resultado[i] = "VERDE";
        }
        else if (palabraSecreta.contains(String.valueOf(letraIntento)))
        {
            resultado[i] = "AMARILLO";
        }
        else
        {
            resultado[i] = "GRIS";
        }
    }

    return resultado;
}

```

```

public String palabraAleatoriaDesdeRecurso(String rutaRecurso) {
    List<String> palabras = new ArrayList<>();

    try (InputStream is = InterfazInicio.class.getResourceAsStream(rutaRecurso)) {
        if (is == null) {
            throw new IllegalArgumentException("No se encontró el recurso: " + rutaRecurso);
        }

        try (BufferedReader br = new BufferedReader(new InputStreamReader(is, StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (!line.isEmpty()) {
                    palabras.add(line);
                }
            }
        }
    } catch (Exception ex) {
        throw new RuntimeException("Error leyendo el recurso: " + rutaRecurso, ex);
    }

    if (palabras.isEmpty()) {
        throw new IllegalStateException("El archivo está vacío: " + rutaRecurso);
    }

    int idx = ThreadLocalRandom.current().nextInt(palabras.size());
    return palabras.get(idx);
}

```

```

public String rutaTxtSegunSeleccion(String idiomaElegido, String dificultadElegida) {
    boolean esEspanol = (idiomaElegido != null) && idiomaElegido.startsWith("Español");
    boolean esFacil = (dificultadElegida != null) && dificultadElegida.startsWith("Facil");

    String lang = esEspanol ? "ES" : "EN";
    String dif = esFacil ? "listaPalabrasFaciles" : "listaPalabrasDificiles";

    return "/recursos/" + dif + "_" + lang + ".txt";
}

```

Lo importante de esta clase es que se encarga de generar una palabra secreta obtenida de los archivos.txt, evaluar cada intento del usuario, controlar intentos disponibles, gestionar puntos y tiempo, validar palabras ingresadas y manejar idioma y dificultad. También interactúa con otras clases como Usuario, Palabra y Ranking

Interfaces gráficas

InterfazInicio

La ventana principal al iniciar el proyecto

```

btnInicio.setLabel(iniciarJuego);
btnInicio.setFont(new Font("Luckiest Guy", Font.PLAIN, 20));
btnInicio.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent accion) {
        visualizarInterfazJuego(accion);
    }
});
btnTutorial.setLabel("Tutorial");
btnTutorial.setFont(new Font("Luckiest Guy", Font.PLAIN, 20));
btnTutorial.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent accion) {
        visualizarInterfazTutorial(accion);
    }
});
btnConfig.setLabel(configuracion);
btnConfig.setFont(new Font("Luckiest Guy", Font.PLAIN, 20));
btnConfig.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent accion) {
        visualizarInterfazConfiguracion(accion);
    }
});
btnRanking.setLabel("Ranking");
btnRanking.setFont(new Font("Luckiest Guy", Font.PLAIN, 20));
btnRanking.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent accion) {
        visualizarInterfazRanking(accion);
    }
});

```

Se puede iniciar el juego, acceder a la configuración, ver el ranking y consultar el tutorial
InterfazWungsdle

(Encargada de gestionar toda la interacción del usuario con el juego)

“Captura de teclado” detecta cuando el usuario presiona teclas y las muestra en la grilla

```
for (char c = 'A'; c <= 'Z'; c++) {  
    String key = String.valueOf(c);  
    panelVerde.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).put(KeyStroke.getKeyStroke(key), key);  
    panelVerde.getActionMap().put(key, new AbstractAction() {  
        @Override  
        public void actionPerformed(ActionEvent e) {  
            if (juegoTerminado)  
                return;  
  
            if (filaActual[0] < 6 && colActual[0] < 5 && liberarTeclado == true) {  
                palabraUsuario += key;  
                casillas[filaActual[0]][colActual[0]].setText(key);  
                colActual[0]++;  
            }  
        }  
    });  
}
```

“Backspace” permite borrar las teclas ingresadas por el usuario

```
// ===== BACKSPACE =====  
panelVerde.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).put(KeyStroke.getKeyStroke("BACK_SPACE"), "borrar");  
panelVerde.getActionMap().put("borrar", new AbstractAction() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        if (juegoTerminado)  
            return;  
  
        if (colActual[0] > 0) {  
            colActual[0]--;  
            casillas[filaActual[0]][colActual[0]].setText("");  
            for (int i = 0; i < 5; i++) {  
                casillas[filaActual[0]][i].setBackground(Color.white);  
            }  
        }  
  
        if (!palabraUsuario.isEmpty()) {  
            palabraUsuario = palabraUsuario.substring(0, palabraUsuario.length() - 1);  
        }  
    }  
});
```

“Enter” define lo que ocurre cuando el usuario da ENTER y lo valida

```
btnComenzarJuego.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if ((txfieldNombreUsuario.getText() != null) && (txfieldNombreUsuario.getText() != "")
            && (!txfieldNombreUsuario.getText().isEmpty() && (!txfieldNombreUsuario.getText().isBlank()
                && txfieldNombreUsuario.getText().length() < 10))) {
            wungsdle.crearNombreUsuario(txfieldNombreUsuario.getText());
            liberarTeclado(e);
            panelNombre.setVisible(false);
            System.out.println(wungsdle.getNombreUsuario());
            panelVerde.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).put(KeyStroke.getKeyStroke("ENTER"),
                "enter");
        } else {
            mensajeErrorNombre = wungsdle.getTextoMensajeErrorNombre();
            wungsdle.alertError(mensajeErrorNombre);
            txfieldNombreUsuario.setText("");
            throw new IllegalArgumentException(mensajeErrorNombre);
        }
    }
});
```

“Creación de casillas” se encarga de generar visualmente el tablero del juego

```
final int[] filaActual = { 0 };
final int[] colActual = { 0 };

for (int fila = 0; fila < 6; fila++) {
    for (int col = 0; col < 5; col++) {
        JLabel label = new JLabel("", SwingConstants.CENTER);
        label.setFont(new Font("Luckiest Guy", Font.BOLD, 25));
        label.setBorder(BorderFactory.createLineBorder(Color.GRAY));
        label.setOpaque(true);
        label.setBackground(Color.WHITE);
        casillas[fila][col] = label;
        panelVerde.add(label);
    }
}
```

Define qué pasa cuando el usuario presiona ENTER.


```

panelVerde.getActionMap().put("enter", new AbstractAction() {
    @Override
    public void actionPerformed(ActionEvent e) {

        /*
         * Ahora además de verificar si lo que ingresa el usuario tiene 5 caracteres,
         * verifica si dicha palabra existe entre las casi 2 mil palabras de los TXT
         * (Codigo se encuentra al final de Wungsdle, idealmente hay que modificarlo,
         * funciona pero es copia)
         */
        if (palabraUsuario.length() == 5 && wungsdle.verificarSiExiste(palabraUsuario,
            wungsdle.getIdiomaActual(), wungsdle.getDificultadActual())) {

            String[] resultado = wungsdle.evaluarColorLetra(palabraUsuario.toLowerCase());
            wungsdle.sumarPuntosPorResultado(resultado);
            boolean acerto = true;

            for (int i = 0; i < 5; i++) {

                if (resultado[i].equals("VERDE")) {
                    casillas[filaActual[0]][i].setBackground(Color.GREEN);
                } else if (resultado[i].equals("AMARILLO")) {
                    casillas[filaActual[0]][i].setBackground(Color.YELLOW);
                    acerto = false;
                } else {
                    casillas[filaActual[0]][i].setBackground(Color.LIGHT_GRAY);
                    acerto = false;
                }
            }
        }
    }
}

```

```

// ===== GANA =====
if (acerto) {
    // TIEMPO
    long tiempoFinal = System.currentTimeMillis();
    long latenciaMs = tiempoFinal - tiempoInicio;

    long minutos = (latenciaMs / 1000) / 60;
    long segundos = (latenciaMs / 1000) % 60;

    wungsdle.setTiempoRespuesta(latenciaMs);
    // punto si gana
    wungsdle.sumarPuntosGanador();
    // guardar
    wungsdle.guardarResultado();
    juegoTerminado = true;

    panelVerde.setEnabled(false);
    panelVerde.setFocusable(false);
    panelVerde.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).clear();
    panelVerde.getActionMap().clear();

    palabraUsuario = "";

    SwingUtilities.invokeLater(() -> {
        InterfazFinal fin = new InterfazFinal(InterfazWungsdle.this, wungsdle.getNombreUsuario(),
            wungsdle.getPalabraSecreta(), true, wungsdle);
        fin.setVisible(true);
    });
}

// ===== PIERDE =====
if (wungsdle.consultarIntentoUsuario() > 1) {
    wungsdle.descontarIntento();
    intentos.setText("INTENTOS: " + wungsdle.consultarIntentoUsuario());

    filaActual[0]++;
    colActual[0] = 0;
    palabraUsuario = "";
} else {

    long tiempoFinal = System.currentTimeMillis();
    long latencia = tiempoInicio - tiempoFinal;
    System.out.println(latencia);
    wungsdle.setTiempoRespuesta(latencia);
    juegoTerminado = true;
    wungsdle.guardarResultado();
    panelVerde.setEnabled(false);
    panelVerde.setFocusable(false);
    panelVerde.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).clear();
    panelVerde.getActionMap().clear();

    SwingUtilities.invokeLater(() -> {
        InterfazFinal fin = new InterfazFinal(InterfazWungsdle.this, wungsdle.getNombreUsuario(),
            wungsdle.getPalabraSecreta(), false, wungsdle);
        fin.setVisible(true);
    });
}
} else {
    for (int i = 0; i < 5; i++) {
        casillas[filaActual[0]][i].setBackground(Color.RED);
    }
}
}

```



Este bloque gestiona el ingreso de palabras. Valida la entrada del usuario, consulta la lógica del juego para evaluar el intento y actualiza visualmente el tablero pintando las casillas según el resultado (correcto, posición incorrecta o incorrecto).

Dificultades encontradas

Durante el desarrollo del trabajo práctico, fuimos debatiendo las diferentes soluciones que cada integrante proponía para el problema planteado. Una de las grandes dificultades que tuvimos que superar, fue obtener un buen diseño que nos permitiera no sólo completar lo requerido, sino también mejorar la calidad general del proyecto.

En una primera etapa, **InterfazInicio** no solo se encargaba de mostrar información al usuario, sino que también gestionaba datos del jugador y lógica que correspondían al funcionamiento del juego. Esto provocaba que una clase de interfaz tuviera responsabilidades que no le correspondían, generando confusión en el código y dificultades en implementaciones posteriores.

Para solucionar este inconveniente, se replanteó el diseño del sistema, definiendo que la clase **Wungsdle** debía ser la encargada de centralizar toda la lógica del juego y manejo de datos

Cambios realizados

- Se eliminó la lógica que se encontraba en la interfaz gráfica, trasladándose a una nueva clase denominada **LogicaPalabra**, encargada de gestionar la validación de palabras, el cálculo de resultados y el estado de la partida. De esta forma, la interfaz quedó limitada exclusivamente a la visualización e interacción con el usuario.
- Se corrigió el algoritmo de evaluación de letras, evitando que una misma letra sea contabilizada más veces de las que aparece en la palabra secreta. Además, se implementó un método de validación que asegura que las palabras ingresadas tengan exactamente cinco letras y existan en el listado correspondiente, evitando fallos en la ejecución.

- Se depuraron los archivos de palabras eliminando aquellas que no cumplían con la longitud requerida, garantizando que todas las partidas sean resolubles.
- Se incorporaron mejoras funcionales, como la visualización del tiempo y puntaje en la interfaz, la inclusión de efectos de sonido y música de fondo, y la modificación del criterio de ordenamiento del ranking considerando el tiempo del jugador.
- Se añadió un ejecutable del juego para facilitar su uso sin necesidad de un entorno de desarrollo.

Conclusión

En esta reentrega se logró mejorar significativamente la calidad del sistema, corrigiendo errores críticos detectados en la versión anterior y optimizando la estructura del proyecto. Se reorganizó el diseño separando la lógica del juego de la interfaz gráfica. Asimismo, se solucionaron problemas de validación de palabras y en la evaluación de letras, garantizando el funcionamiento correcto del juego. Finalmente se incorporaron mejoras para la experiencia del usuario, como la visualización del tiempo y puntaje, junto con la integración de efectos de sonido.